

Comparative Analysis of Deep Learning Frameworks

Abhinav Kumar

Roll No. 59 PRN No. 2414110730

Rishikesh Yadav

Roll No. 32 PRN No. 2414110703

Department of Electronics and Communication Engineering

Bharati Vidyapeeth College of Engineering, Pune, India

Abstract

Deep learning frameworks provide the software infrastructure required to express neural-network computations and execute them efficiently on modern hardware accelerators. This paper presents a practical comparative study of PyTorch and TensorFlow through the Keras API using an identical convolutional neural network (CNN) trained on the CIFAR-10 image-classification dataset. The experiment is designed to isolate framework-level characteristics by keeping the principal workload variables constant. Both implementations use the same two-layer CNN, five training epochs, a batch size of 64, the Adam optimizer, sparse categorical crossentropy loss, and an NVIDIA T4 Tensor Core GPU in Google Colab. The comparison evaluates training time, peak GPU memory allocation, and development effort. TensorFlow/Keras completes the reported five-epoch workload in 26.79 seconds, whereas PyTorch requires 71.67 seconds. Conversely, PyTorch reports a peak GPU memory allocation of 46.82 MB compared with 851.62 MB for TensorFlow/Keras. The implementation effort is also different, with approximately 20 core lines for TensorFlow/Keras and approximately 45 for PyTorch. The results demonstrate that the frameworks make different architectural and engineering trade-offs between execution speed, memory behavior, abstraction, and programmer control. The study does not claim a universally superior framework; instead, it provides an experimentally grounded basis for selecting a framework according to application requirements.

Keywords: deep learning, PyTorch, TensorFlow, Keras, convolutional neural network, CIFAR-10, GPU memory, training time, framework comparison

1 Introduction

Deep learning has become a central approach for computer vision, natural language processing, speech processing, and many other machine-learning tasks. The practical development of deep-learning systems depends not only on neural-network architectures but also on software frameworks that manage tensors, automatic differentiation, hardware acceleration, and model training. PyTorch and TensorFlow are two major frameworks used to construct and execute such systems.

Although both frameworks can implement the same mathematical model, they may differ in execution behavior and programming experience. These differences become relevant when a developer must consider training speed, GPU-memory usage, implementation effort, debugging, or the ability to customize the training procedure.

The objective of this study is to empirically compare the two frameworks using the same CNN workload. Instead of comparing classification accuracy, the study concentrates on system-level metrics. This choice is important because an identical architecture trained with the same data and hyperparameters can produce broadly comparable learning behavior, while framework-level execution characteristics may still differ.

1.1 Objectives

The primary objectives are: (1) to implement an equivalent CNN in TensorFlow/Keras and PyTorch; (2) to execute both implementations on the same NVIDIA T4 GPU; (3) to measure total training time; (4) to measure peak GPU memory allocation; (5) to compare implementation effort; and (6) to interpret the observed differences in terms of framework execution and memory-management strategies.

1.2 Contributions

The contributions of this work are a controlled two-framework benchmark, a quantitative comparison of execution and memory behavior, a qualitative comparison of programming abstractions, and a discussion of the practical situations in which each framework may be preferable.

2 Background and Related Concepts

A deep-learning framework provides abstractions for tensors, neural-network layers, automatic differentiation, optimizers, data loading, and accelerator execution. In a typical supervised image-classification task, the input batch is passed through a network, a loss value is calculated from the predictions and labels, gradients are obtained through backpropagation, and an optimizer updates the trainable parameters.

2.1 Convolutional Neural Networks

A convolutional neural network is particularly suitable for image data because convolutional layers learn spatially local patterns. Early layers can learn simple features while later layers combine them into increasingly complex representations. The benchmark uses a two-layer CNN followed by fully connected dense layers. Because the same architecture is implemented in both frameworks, the intended computational workload remains comparable.

2.2 TensorFlow and Keras

TensorFlow provides an ecosystem for tensor computation and machine learning, while Keras provides a high-level API for constructing and training models. In the project implementation, the Keras API represents the higher-abstraction side of the comparison. Layers can be stacked to form the model and the standard training process can be initiated using the `model.fit()` interface.

This abstraction reduces the amount of explicit training-loop code. Forward computation, loss evaluation, gradient calculation, and parameter updates can be managed by the framework. The project therefore associates TensorFlow/Keras with concise implementation and rapid prototyping.

2.3 PyTorch

PyTorch follows a more explicit Python programming model. The developer can directly control model execution, tensor devices, gradient management, and optimization. In the project implementation, operations such as moving tensors to the GPU using `.to(device)`, clearing gradients, calculating the loss, and invoking `loss.backward()` are exposed to the programmer.

The explicit design increases the amount of code but provides granular control over the training pipeline. This characteristic is useful when standard training behavior must be changed or when an experiment requires direct manipulation of intermediate operations.

2.4 System-Level Comparison

The present work treats the frameworks as engineering systems rather than merely APIs. The important question is not which syntax is easier in isolation, but how the selected abstractions affect execution time, memory allocation, and development effort when the same workload is executed.

3 Experimental Setup

A controlled experiment was designed to minimize factors unrelated to the framework. The two implementations use the same dataset, architecture, training duration, batch size, optimizer, loss function, and reported hardware environment. The models are trained sequentially in isolated runtime environments to reduce the possibility of memory left by one framework influencing the other measurement.

Property	Configuration
Cloud environment	Google Colab
GPU	NVIDIA T4 Tensor Core GPU
Dataset	CIFAR-10
Dataset size	60,000 color images
Image resolution	32 × 32
Number of classes	10
Architecture	2-layer CNN + fully connected dense layers
Epochs	5
Batch size	64
Optimizer	Adam
Loss	Sparse categorical crossentropy

Table 1: Main experimental configuration.

3.1 Dataset

CIFAR-10 contains 60,000 color images with a spatial resolution of 32×32 pixels distributed across 10 classes. It is a compact and widely used image-classification benchmark and is suitable for a controlled CNN experiment because the data can be processed rapidly while still exercising convolutional and dense layers.

3.2 Model and Hyperparameters

The project uses a two-layer CNN followed by fully connected dense layers. Both implementations use five epochs and a batch size of 64. Adam is selected as the optimizer and sparse categorical crossentropy is used as the loss function. These values are kept constant across the two implementations.

3.3 Measurement Procedure

Training time is measured using Python's built-in time functionality. The project records peak GPU memory using `torch.cuda.max_memory_allocated` for PyTorch and `tf.config.experimental.get_memory_info` for TensorFlow. The measurements therefore represent the framework-specific diagnostic values reported by the project rather than a manually estimated model size.

4 Quantitative Results

The benchmark produces clear differences across the three selected metrics. TensorFlow/Keras provides the faster reported execution and the smaller code footprint, while PyTorch reports substantially lower peak GPU memory allocation.

Evaluation metric	TensorFlow / Keras	PyTorch	Reported comparison
Total training time	26.79 s	71.67 s	TF $\approx 2.67\times$ faster
Peak GPU memory	851.62 MB	46.82 MB	PT $\approx 94.5\%$ less
Core implementation code	≈ 20 lines	≈ 45 lines	PT $\approx 125\%$ more

Table 2: Quantitative benchmark results.

4.1 Training Time

TensorFlow/Keras completes the five-epoch training run in 26.79 seconds, whereas PyTorch requires 71.67 seconds. The reported difference is 44.88 seconds over the complete five-epoch run. On the basis of the project measurements, TensorFlow is approximately 2.67 times faster for this workload.

4.2 GPU Memory

TensorFlow/Keras records 851.62 MB of peak GPU memory allocation, while PyTorch records 46.82 MB. The reported PyTorch value is approximately 94.5% lower. This is the largest resource-footprint difference in the experiment.

4.3 Development Effort

The TensorFlow/Keras implementation is reported to require approximately 20 core lines, while the PyTorch implementation requires approximately 45. The project therefore reports approximately 125% more core implementation code for PyTorch. The difference reflects programming abstraction rather than a direct measure of software quality.

4.4 Summary of Observations

The benchmark can be summarized as follows: TensorFlow/Keras is the fastest and uses the least core code in the reported experiment; PyTorch has the lowest reported peak GPU allocation and provides a more explicit training interface.

5 Technical Analysis of Training-Time Behavior

5.1 Execution and Graph Processing

The project attributes the training-time difference primarily to the execution model used by the two frameworks. Under the Keras training interface, TensorFlow can transform a high-level training procedure into optimized operations before execution. This reduces the amount of explicit Python-level coordination required during the training loop.

The PyTorch implementation uses an explicit Python training loop. Each batch is processed through visible stages such as forward computation, loss calculation, gradient computation, and optimizer update. While this approach gives the programmer control, the additional Python-level orchestration can introduce overhead in a small benchmark where the GPU computation itself is relatively lightweight.

5.2 Epoch-Level Interpretation

Dividing the reported totals by five gives approximately 5.36 seconds per epoch for TensorFlow/Keras and 14.33 seconds per epoch for PyTorch. These values are descriptive of the measured run. They should not be interpreted as universal performance constants because execution time can change with model depth, batch size, data pipeline, software version, GPU utilization, and other runtime conditions.

5.3 Why Faster Does Not Mean Universally Better

A faster training run is beneficial when the primary objective is rapid iteration, repeated experimentation, or high-throughput model development. However, training speed is only one engineering criterion. If memory is severely constrained, a framework with lower measured allocation may be preferable even if it requires more time to complete the same workload.

5.4 Practical Interpretation

For the exact workload tested, TensorFlow/Keras provides a clear execution-time advantage. The result supports its use when standard training workflows and rapid iteration are important. It does not establish that TensorFlow will outperform PyTorch on every architecture or hardware configuration.

6 GPU Memory Allocation Analysis

6.1 Memory Allocation Difference

The peak GPU memory measurements differ by approximately 804.80 MB: 851.62 MB for TensorFlow/Keras versus 46.82 MB for PyTorch. The magnitude of this difference makes memory behavior a central part of the benchmark rather than a secondary observation.

6.2 TensorFlow Memory Strategy

The project characterizes TensorFlow's behavior as more aggressive memory mapping or pre-allocation. Claiming a larger memory region can reduce repeated allocation requests while a workload is executing. This strategy may contribute to efficient execution but produces a substantially larger reported peak allocation in the experiment.

6.3 PyTorch Memory Strategy

The project characterizes PyTorch as dynamically requesting memory required by the current workload. The reported value is consequently much smaller. This behavior is presented as evidence of a lightweight memory footprint for the selected CNN and batch configuration.

6.4 Memory Metrics Require Context

Peak allocated memory is not identical to total physical GPU capacity or necessarily to the amount of memory actively occupied by useful tensors at every moment. Different frameworks expose different memory diagnostics and may reserve memory for allocator efficiency. Therefore, the numerical values should be interpreted as measurements produced by the selected framework-specific methods.

6.5 Implications for Resource-Constrained Systems

Lower memory allocation can be important when several workloads share a GPU or when experimentation is performed on a device with limited VRAM. The reported PyTorch result therefore represents a significant practical advantage for memory-sensitive environments, even though the same experiment records slower execution.

7 Development Effort and Programming Abstraction

7.1 TensorFlow/Keras Programming Model

The Keras API provides a high-level model-building and training interface. Sequential layer construction makes the architecture concise, while `model.fit()` hides many details of the training procedure. The project reports approximately 20 core lines for the implementation.

This level of abstraction can be useful for rapid prototyping because developers spend less time writing boilerplate training logic. It also reduces the number of individual operations that must be managed manually in a standard experiment.

7.2 PyTorch Programming Model

PyTorch exposes more of the training pipeline. The developer must explicitly manage the model, device placement, gradient clearing, forward pass, loss calculation, backward pass, and optimizer update. The project reports approximately 45 core lines for the implementation.

The additional code should not be interpreted simply as inefficiency. Explicit code can make intermediate operations easier to inspect and modify. For research workflows, custom architectures, unusual losses, or experimental training algorithms, this control can be valuable.

7.3 Abstraction Versus Control

The benchmark demonstrates a common engineering trade-off: higher abstraction reduces implementation effort, while lower abstraction can provide greater control. TensorFlow/Keras is advantageous when the developer wants a concise standard workflow. PyTorch is advantageous when the developer needs direct access to individual stages of training.

7.4 Code Complexity as a Measured Dimension

The line-count comparison is useful as a project-level indicator but should not be treated as a complete measure of maintainability. Software quality also depends on clarity, testing, documentation, extensibility, debugging, and the complexity of the underlying task.

8 Comparative Discussion

The experiment reveals that the two frameworks optimize for different priorities. TensorFlow/Keras combines a high-level training interface with the fastest reported execution and the smallest core-code footprint. PyTorch combines explicit programming control with the lowest reported peak GPU allocation.

Aspect	TensorFlow / Keras	PyTorch
Training speed	Advantage in this experiment	Lower in this experiment
Reported peak memory	Higher	Much lower
Core code	≈20 lines	≈45 lines
Abstraction	High	Explicit
Training-loop control	Lower-level details hidden	Fine-grained control
Best fit suggested by project	Rapid prototyping / standard workflows	Research / custom experimentation

Table 3: Qualitative comparison derived from the benchmark.

8.1 Standard Product-Oriented Workflows

When a development team uses a conventional model architecture and values rapid implementation and training throughput, TensorFlow/Keras offers practical advantages in the reported experiment. The high-level API reduces boilerplate and the measured execution time is substantially shorter.

8.2 Research and Custom Workflows

When the developer needs to modify the training loop or inspect individual operations, PyTorch's explicit programming model can be valuable. The additional code provides direct access to model and optimization steps, supporting experimentation and debugging.

8.3 Memory-Constrained Workflows

The reported memory result changes the framework-selection picture. A system with tight GPU-memory limits may value PyTorch's measured 46.82 MB allocation more strongly than the speed advantage measured for TensorFlow. This illustrates why framework selection should consider the complete system rather than a single benchmark metric.

8.4 Overall Trade-off

The benchmark does not identify a single winner. Instead, it establishes a trade-off between execution speed, measured memory footprint, code volume, and control. The most appropriate framework depends on which of these factors is dominant in the intended application.

9 Reproducibility and Experimental Procedure

Reproducibility is important for a framework comparison because small differences in hardware state, runtime configuration, or measurement procedure can affect results. The project provides a compact procedure that can be repeated using the same dataset, model, hyperparameters, and GPU environment.

9.1 Reproduction Steps

The experimental workflow can be summarized as: (1) initialize a Google Colab runtime with an NVIDIA T4 GPU; (2) load the CIFAR-10 dataset; (3) construct the same two-layer CNN in TensorFlow/Keras; (4) train for five epochs with batch size 64, Adam, and sparse categorical crossentropy; (5) record elapsed time and peak GPU memory; (6) reset or isolate the runtime; (7) implement the equivalent CNN in PyTorch; (8) repeat the same training configuration; and (9) compare the resulting measurements.

9.2 Measurement Isolation

The models were trained sequentially in isolated runtime environments to reduce cross-contamination of GPU memory. This is particularly important for the memory comparison because a previous framework can reserve or cache GPU memory that might otherwise influence a subsequent measurement.

9.3 Reported Diagnostic Functions

For PyTorch, the project uses `torch.cuda.max_memory_allocated`. For TensorFlow, it uses `tf.config.experimental.get_memory_info`. Timing is obtained using Python's built-in time module. Because the memory APIs differ, the results should be understood as framework-specific peak allocation measurements rather than a single universal memory definition.

9.4 Reproducibility Considerations

For stronger future reproducibility, the software versions, CUDA/cuDNN versions, random seeds, number of timing repetitions, warm-up procedure, and exact model layer parameters should also be recorded. Repeating the benchmark multiple times would allow averages and variability estimates to be reported rather than relying on a single measured run.

10 Limitations

The benchmark provides useful evidence for the selected workload but has several limitations that restrict the generality of its conclusions.

10.1 Hardware and Runtime

Only an NVIDIA T4 Tensor Core GPU in Google Colab is reported. Results may differ on other NVIDIA GPUs, different accelerator generations, CPUs, or other cloud environments. Runtime software versions can also influence execution and memory behavior.

10.2 Model Complexity

The experiment uses a two-layer CNN followed by dense layers. Larger architectures, residual networks, transformers, recurrent networks, or models with substantially different memory patterns may produce different framework trade-offs.

10.3 Dataset and Batch Size

The benchmark uses CIFAR-10 and a batch size of 64. Larger image resolutions, more complex preprocessing, or different batch sizes can change the balance between CPU overhead, GPU utilization, and memory allocation.

10.4 Training Duration

Only five epochs are used. Longer experiments may expose different initialization, caching, or sustained-throughput behavior. The current values are therefore a case-study measurement rather than a long-term performance characterization.

10.5 Single-Run Measurements

The project reports the measured values but does not establish a large repeated-sample statistical analysis. Future studies should repeat each condition several times and report mean, standard deviation, and confidence intervals where appropriate.

10.6 Memory Interpretation

Framework-specific memory allocation statistics are not interchangeable with physical VRAM consumption in every context. The reported 851.62 MB and 46.82 MB values should therefore be interpreted in relation to the measurement APIs used by the two frameworks.

11 Future Work

The current benchmark can be expanded into a broader framework-evaluation study. First, multiple CNN architectures of increasing depth could be evaluated to determine whether the training-time and memory trends remain consistent. Second, batch size could be varied systematically to observe how memory allocation scales. Third, the experiment could be repeated on different GPU classes and CPU-only systems.

A second extension would evaluate additional metrics, including data-loading time, GPU utilization, host-memory usage, inference latency, model initialization time, and energy consumption. These metrics could provide a more complete systems-level view of framework efficiency.

Another useful extension would compare several TensorFlow and PyTorch execution configurations rather than only the standard project implementations. For example, future work could investigate different compilation or execution modes, optimized data pipelines, and mixed-precision training. Such experiments would help separate framework defaults from the maximum performance available through each ecosystem.

Finally, repeated experiments with controlled random seeds and multiple runs could provide statistically stronger conclusions. A larger benchmark suite involving several datasets and model families would allow researchers to determine whether the observed trade-offs generalize beyond CIFAR-10 and the selected two-layer CNN.

11.1 Proposed Extended Benchmark

Metric	Current study	Suggested extension
Training time	One reported run	Multiple repetitions + variance
GPU memory	Peak allocation	Allocated + reserved + utilization
Models	2-layer CNN	CNN, ResNet, transformer
Hardware	NVIDIA T4	Multiple GPU generations
Data	CIFAR-10	CIFAR-10 + larger datasets
Precision	Standard	FP32 + mixed precision

Table 4: Suggested extensions to the benchmark.

12 Conclusion

This paper presented a controlled empirical comparison of PyTorch and TensorFlow/Keras using an identical CNN workload on CIFAR-10. The experiment was performed in Google Colab on an NVIDIA T4 Tensor Core GPU using five epochs, a batch size of 64, Adam optimization, and sparse categorical crossentropy. The study focused on training time, peak GPU memory allocation, and development effort rather than model accuracy.

The measured results show a strong execution-time advantage for TensorFlow/Keras: 26.79 seconds compared with 71.67 seconds for PyTorch, or approximately 2.67 times faster for the selected workload. The memory result moves in the opposite direction. PyTorch reports 46.82 MB compared with TensorFlow/Keras at 851.62 MB, representing approximately 94.5% lower reported peak allocation. Development effort also differs, with approximately 20 core lines for TensorFlow/Keras and approximately 45 for PyTorch.

The findings support a trade-off interpretation. TensorFlow/Keras provides a concise, highly abstracted interface and faster measured execution in the benchmark. PyTorch provides a more explicit training workflow and substantially lower reported peak GPU allocation. The additional PyTorch code provides greater direct control over training operations and can therefore be valuable in research and custom experimentation.

The most important conclusion is that framework selection should be driven by system requirements rather than by a universal ranking. For rapid prototyping and standard workloads resembling the experiment, TensorFlow/Keras has clear advantages in the measured dimensions of speed and code volume. For memory-sensitive or highly customizable training workflows, PyTorch offers important advantages. The benchmark therefore demonstrates different engineering priorities rather than a single universally superior framework.

13 Author Information

Abhinav Kumar — Roll No. 59 — PRN No. 2414110730

Rishikesh Yadav — Roll No. 32 — PRN No. 2414110703

Department of Electronics and Communication Engineering, Bharati Vidyapeeth College of Engineering, Pune, India.

Acknowledgments

The authors acknowledge the project report and experimental material supplied for this study, as well as the open-source deep-learning ecosystem used to implement the benchmark. The paper is prepared as an academic research-paper version of the project work.

References

- [1] Project Report, Comparative Analysis of Deep Learning Frameworks: Project Report—Framework Benchmark, Department of Electronics and Communication Engineering, Bharati Vidyapeeth College of Engineering.
- [2] F. Chollet, Deep Learning with Python, 2nd ed., Manning Publications, 2021.
- [3] A. Paszke et al., PyTorch: An Imperative Style, High-Performance Deep Learning Library, Advances in Neural Information Processing Systems, 2019.
- [4] M. Abadi et al., TensorFlow: A System for Large-Scale Machine Learning, Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation, 2016.
- [5] A. Krizhevsky, Learning Multiple Layers of Features from Tiny Images, University of Toronto, 2009.
- [6] The CIFAR-10 dataset documentation and associated benchmark resources.
- [7] TensorFlow and Keras documentation for model training and GPU execution.
- [8] PyTorch documentation for CUDA execution and GPU memory diagnostics.